meddddgo

meddddgo,顾名思义，go语言

Address	Length	Type	String
.rdata:0···	00000005	C	TFlag
.rdata:0···	0000000B	C	\n*abi.TFlag
.rdata:0···	0000000E	C	\r*abi.FuncFlag
.rdata:0···	00000016	C	\\??netChanSendflagtyp_
.rdata:0···	00000005	C	flag{
.rdata:0···	00000008	C	rflags
.rdata:0···	00000339	C	GODEBUG: value \"DuplicateTokenExCreateNamedPipeWGetCurrentThreadGetModuleHandleWRtlVirtualUnwindallowmultipl
.rdata:0···	00000029	C	Congratulate! The mad go flag is flag{%s}
.rdata:0···	0000002A	C	bytes; incompatible with mutex flag mask
.rdata:0···	00000022	C	runtime.(*scavChunkFlags).isEmpty
.rdata:0···	00000023	C	runtime.(*scavChunkFlags).setEmpty
.rdata:0···	00000026	C	runtime.(*scavChunkFlags).setNonEmpty
.rdata:0···	00000029	C	runtime.profIndex.addCountsAndClearFlags
.rdata:0···	0000001C	C	gAehhfV95/cpuflags_amd64.go
.rdata:0···	00000010	C	Yn5CLZfLagBi.go
.idata:0···	0000000C	C	WerSetFlags
.idata:0···	0000000C	C	WerGetFlags

找到主函数

```
23  sub_140048420();
24  sub_140048420();
25  sub_140048420();
26  v15 = &unk_1400C1A80;
27  v16 = &off_14010A740;
28  sub_1400AA600();
29  v5 = qword_1401C8820;
30  v14 = v1;
31  sub_1400735B0(v13, 1);
32  runtime_makeslice();
33  v9 = v1;
34  *(_QWORD *)&v9 = sub_1400735B0(&v5, 1);
35  *((_QWORD *)&v9 + 1) = 4096;
36  n4096 = 4096;
37  v11 = &off_14010AE08;
38  v12 = v5;
39  v13[3] = -1;
40  v13[4] = -1;
41  *(_QWORD *)&v14 = v9;
42  sub_14007391A((char *)&v14 + 8, (char *)&v9 + 8);
43  sub_14007DF20();
44  if ( (unsigned __int8)sub_1400B3740() )
45  {
46    v6 = v1;
47    v2 = sub_14006AC40();
48    *(_QWORD *)&v6 = &unk_1400C1A80;
49    *((_QWORD *)&v6 + 1) = v2;
50    return sub_1400AA4E0(41, &v6, v3, "Congratulate! The mad go flag is flag{%s}", (const char *)1);
51  }
52  else
53  {
54    v7 = &unk_1400C1A80;
55    v8 = &off_14010A750;                        // "You go mad, right?"
56    return sub_1400AA6E0();
57  }
58 }
```

找到验证函数

```
_BOOL8 __fastcall sub_1400B3740()
{
  __int64 v0; // rax
  __int64 n2; // rbx
  __int64 v2; // r14
  char n10; // r9
  char v4; // cl
  char v5; // cl
  __int64 v6; // r8
  __int64 v7; // rdx
  void *v8; // rdi
  __int64 v9; // rax
  __int64 n16; // rcx
  unsigned __int8 v11; // si
  _UNKNOWN *retaddr; // [rsp+20h] [rbp+8h] BYREF
  __int64 v14; // [rsp+28h] [rbp+10h]

  if ( (unsigned __int64)&retaddr <= *(_QWORD *)(v2 + 16) )
    sub_140071040();
  if ( n2 >= 2 && *(_BYTE *)(n2 + v0 - 2) == 13 && *(_BYTE *)(v0 + n2 - 1) == 10 )
  {
    n2 -= 2;
  }
  else if ( n2 )
  {
    n10 = *(_BYTE *)(v0 + n2 - 1);
    if ( n10 == 10 || n10 == 13 )
      --n2;
  }
  v14 = v0;
  if ( n2 >= 5 )
  {
    v4 = sub_140003100();
    v0 = v14;
  }
  else
```

魔改SM4

```
__int64 __fastcall sub_1400B3540(__int64 a1, unsigned __int64 a2, _QWORD a3, __int64 n16, unsigned __int64 a5)
{
  __int64 v5; // r14
  __int64 result; // rax
  signed __int64 v7; // rcx
  unsigned __int64 v8; // rdx
  __int64 n16_1; // r8
  __int64 v10; // r9
  unsigned __int64 i; // rbx
  __int128 *v12; // r9
  __int128 v13; // [rsp+0h] [rbp-68h]
  __int128 v14; // [rsp+40h] [rbp-28h] BYREF
  unsigned __int64 v15; // [rsp+50h] [rbp-18h]
  unsigned __int64 v16; // [rsp+58h] [rbp-10h]
  __int64 v17; // [rsp+60h] [rbp-8h]
  _UNKNOWN *retaddr; // [rsp+70h] [rbp+8h] BYREF

  if ( (unsigned __int64)&retaddr <= *(_QWORD *)(v5 + 16) )
    sub_140071040();
  if ( (a2 & 0xF) != 0 )
    return 0;
  result = runtime_makeslice();
  v17 = result;
  v7 = a2;
  v8 = a5;
  n16_1 = n16;
  v10 = a1;
  for ( i = 0; v7 > (__int64)i; i = v16 )
  {
    if ( v8 < i + 16 )
      sub_140073380();
    if ( i > i + 16 )
      sub_1400733C0();
    v15 = i;
    v16 = i + 16;
    sub_1400B32A0(v10 + (i & ((__int64)(i - v8) >> 63)), 16, v10 + (i & ((__int64)(i - v8) >> 63)), n16_1, v8 - i);
```

```
IDA View-A    Pseudocode-B    Pseudocode-A    Strings    Hex View-1    Local Types    Impor
__int64 __fastcall sub_1400B32A0(__int64 a1, __int64 n16, __int64 a3, __int64 a4, unsigned __int64 a5)
{
  __int64 v5; // r14
  __int64 n4; // rax
  unsigned __int64 v7; // rbx
  __int64 n32; // rax
  unsigned int v9; // edx
  int v10; // esi
  __int64 result; // rax
  char v12[8]; // [rsp+10h] [rbp-1B8h] BYREF
  __int64 v13; // [rsp+18h] [rbp-1B0h]
  __int64 v14; // [rsp+20h] [rbp-1A8h]
  __int64 v15; // [rsp+28h] [rbp-1A0h]
  __int64 v16; // [rsp+30h] [rbp-198h]
  __int64 v17; // [rsp+38h] [rbp-190h]
  __int64 v18; // [rsp+40h] [rbp-188h]
  __int64 v19; // [rsp+48h] [rbp-180h]
  __int64 v20; // [rsp+50h] [rbp-178h]
  __int64 v21; // [rsp+58h] [rbp-170h]
  __int64 v22; // [rsp+60h] [rbp-168h]
  __int64 v23; // [rsp+68h] [rbp-160h]
  __int64 v24; // [rsp+70h] [rbp-158h]
  __int64 v25; // [rsp+78h] [rbp-150h]
  __int64 v26; // [rsp+80h] [rbp-148h]
  __int64 v27; // [rsp+88h] [rbp-140h]
  _QWORD v28[4]; // [rsp+90h] [rbp-138h] BYREF
  unsigned __int32 v29; // [rsp+B0h] [rbp-118h]
  int v30; // [rsp+B4h] [rbp-114h]
  _DWORD v31[20]; // [rsp+B8h] [rbp-110h] BYREF
  unsigned int v32[46]; // [rsp+108h] [rbp-C0h] BYREF
  unsigned __int64 v33; // [rsp+1C0h] [rbp-8h]

  if ( (unsigned __int64)v28 <= *(_QWORD *)(v5 + 16) )
```

魔改处，多加了一个异或

```
      sub_140073300();
    *((_DWORD *)v47 + n4_3) = _byteswap_ulong(*(_DWORD *)(v29 + (((__int64)(v35 - v30) >> 63) & v35)));
  }
  sub_140073599(&v45);
  for ( n4_1 = 0; n4_1 < 4; ++n4_1 )
    *((_DWORD *)&v47[1] + n4_1) = *((_DWORD *)v47 + n4_1) ^ dword_1401B93C0[n4_1] ^ dword_1401B93D0[n4_1];
  sub_1400735A2(v44);
  for ( n32 = 0; n32 < 32; ++n32 )
  {
    v38 = dword_1401B9A00[n32]
        ^ *((_DWORD *)&v47[1] + n32 + 3)
        ^ *((_DWORD *)&v47[1] + n32 + 2)
        ^ *((_DWORD *)&v47[1] + n32 + 1);
    v39 = byte_1401B9EE0[LOBYTE(dword_1401B9A00[n32])
```

```
BF                      ^ db ' 66h ' ; f
C0 ; _DWORD dword_1401B93C0[4]
C0 dword_1401B93C0 dd 0A3B1BAC6h, 56AA3350h, 677D9197h, 0B27022DCh
C0                                                      ; DATA XREF: sub_1400B3020+E5↑o
D0 ; _DWORD dword_1401B93D0[4]
D0 dword_1401B93D0 dd 0A5A5A5A5h, 3C3C3C3Ch, 5A5A5A5Ah, 0C3C3C3C3h
D0                                                      ; DATA XREF: sub_1400B3020:loc_1400B
E0 unk_1401B93E0   db    0                              ; DATA XREF:  .data:0000000140 1B9CD0↓
```

S盒

```
byte_1401B9EE0  db 0D6h, 90h, 0E9h, 0FEh, 0CCh, 0E1h, 3Dh, 0B7h, 16h, 0B6h
                                        ; DATA XREF: sub_1400B3020+14D↑o
                                        ; sub_1400B32A0+130↑o
                db 14h, 0C2h, 28h, 0FBh, 2Ch, 5, 2Bh, 67h, 9Ah, 76h, 2Ah
                db 0BEh, 4, 0C3h, 0AAh, 44h, 13h, 26h, 49h, 86h, 6, 99h
                db 9Ch, 42h, 50h, 0F4h, 91h, 0EFh, 98h, 7Ah, 33h, 54h
                db 0Bh, 43h, 0EDh, 0CFh, 0ACh, 62h, 0E4h, 0B3h, 1Ch, 0A9h
                db 0C9h, 8, 0E8h, 95h, 80h, 0DFh, 94h, 0FAh, 75h, 8Fh
                db 3Fh, 0A6h, 47h, 7, 0A7h, 0FCh, 0F3h, 73h, 17h, 0BAh
                db 83h, 59h, 3Ch, 19h, 0E6h, 85h, 4Fh, 0A8h, 68h, 6Bh
                db 81h, 0B2h, 71h, 64h, 0DAh, 8Bh, 0F8h, 0EBh, 0Fh, 4Bh
                db 70h, 56h, 9Dh, 35h, 1Eh, 24h, 0Eh, 5Eh, 63h, 58h, 0D1h
                db 0A2h, 25h, 22h, 7Ch, 3Bh, 1, 21h, 78h, 87h, 0D4h, 0
                db 46h, 57h, 9Fh, 0D3h, 27h, 52h, 4Ch, 36h, 2, 0E7h, 0A0h
                db 0C4h, 0C8h, 9Eh, 0EAh, 0BFh, 8Ah, 0D2h, 40h, 0C7h, 38h
                db 0B5h, 0A3h, 0F7h, 0F2h, 0CEh, 0F9h, 61h, 15h, 0A1h
                db 0E0h, 0AEh, 5Dh, 0A4h, 9Bh, 34h, 1Ah, 55h, 0ADh, 93h
                db 32h, 30h, 0F5h, 8Ch, 0B1h, 0E3h, 1Dh, 0F6h, 0E2h, 2Eh
                db 82h, 66h, 0CAh, 60h, 0C0h, 29h, 23h, 0ABh, 0Dh, 53h
                db 4Eh, 6Fh, 0D5h, 0DBh, 37h, 45h, 0DEh, 0FDh, 8Eh, 2Fh
                db 3, 0FFh, 6Ah, 72h, 6Dh, 6Ch, 5Bh, 51h, 8Dh, 1Bh, 0AFh
                db 92h, 0BBh, 0DDh, 0BCh, 7Fh, 11h, 0D9h, 5Ch, 41h, 1Fh
                db 10h, 5Ah, 0D8h, 0Ah, 0C1h, 31h, 88h, 0A5h, 0CDh, 7Bh
                db 0BDh, 2Dh, 74h, 0D0h, 12h, 0B8h, 0E5h, 0B4h, 0B0h, 89h
                db 69h, 97h, 4Ah, 0Ch, 96h, 77h, 7Eh, 65h, 0B9h, 0F1h
                db 9, 0C5h, 6Eh, 0C6h, 84h, 18h, 0F0h, 7Dh, 0ECh, 3Ah
                db 0DCh, 4Dh, 20h, 79h, 0EEh, 5Fh, 3Eh, 0D7h, 0CBh, 39h
                db 48h
```

初始化函数

```
__int64 __fastcall sub_1400B3940()
{
  __int64 v0; // r14
  __int128 v1; // xmm15
  __int64 v2; // rax
  __int64 *v3; // r11
  __int64 v4; // rcx
  __int64 v5; // rax
  __int64 *v6; // r11
  __int64 v7; // rcx
  __int64 v8; // rax
  __int64 *v9; // r11
  __int64 v10; // rcx
  _QWORD *v11; // rax
  __int64 v12; // rdx
  __int64 *v13; // r11
  __int64 v14; // rcx
  _QWORD *v15; // rax
  __int64 v16; // rdx
  __int64 *v17; // r11
  __int64 v18; // rcx
  _QWORD *v19; // rax
  __int64 v20; // rdx
  __int64 *v21; // r11
  __int64 v22; // rcx
  _QWORD *v23; // rax
  __int64 v24; // rdx
  __int64 *v25; // r11
  __int64 v26; // rcx
```

密钥

```
__int64 __fastcall sub_1400B2F60()
{
  __int128 v0; // xmm15
  __int64 n16; // rax
  __int64 n3; // rax
  __int64 n16_1; // rcx
  __int128 v4; // [rsp+0h] [rbp-10h]

  v4 = v0;
  for ( n16 = 0; n16 < 16; ++n16 )
    *((_BYTE *)&v4 + n16) = __ROL1__(byte_14020E2E0[(5 * (_BYTE)n16 + 3) & 0xF], 1) ^ (11 * n16) ^ 0xA5;
  for ( n3 = 0; n3 < 3; ++n3 )
  {
    for ( n16_1 = 0; n16_1 < 16; ++n16_1 )
      *((_BYTE *)&v4 + n16_1) ^= 17 * n3
                               + *((_BYTE *)&v4 + (((_DWORD)n16_1 + 1) & 0xF))
                               + *((_BYTE *)&v4 + (((_BYTE)n16_1 + 5) & 0xF));
  }
  xmmword_14020E2F0 = v4;
  return n3;
}
```

调试得到， xmmword 26998B3B3292B645F1E3B68D7522A4ADh

```
E0 byte_7FF63B2AE2E0 db 10h, 23h, 45h, 67h, 89h, 0ABh, 0CDh, 0EFh, 1, 35h, 79h, 0BDh, 0F0h
E0                                          ; DATA XREF: sub_7FF63B1526E0:loc_7FF63B152799↑o
E0                                          ; sub_7FF63B1526E0:loc_7FF63B1527DD↑o ...
ED db 22h, 44h, 66h
F0 xmmword_7FF63B2AE2F0 xmmword 26998B3B3292B645F1E3B68D7522A4ADh
F0                                          ; DATA XREF: sub_7FF63B1526E0+D9↑o
F0                                          ; sub_7FF63B1526E0+120↑o ...
00 unk_7FF63B2AE300 db    0                 ; DATA XREF: sub_7FF63B10B8C0+17↑o
00                                          ; sub_7FF63B10B8C0+F6↑o ...
01 db    0
02 db    0
03 db    0
```

比对

```
  n16 = 0;
  v11 = 0;
  while ( n16 < 16 )
  {
    v11 |= *((_BYTE *)off_1401C0310 + n16) ^ *(_BYTE *)(n16 + v9);
    ++n16;
  }
  return ((unsigned int)v11 - 1) >> 31 == 1;
}
```

密文

```
1401B95C0 aPowrprofDll:                           ; DATA XREF: sub_140036EE0+19↑o
1401B95C0                 text "UTF-16LE", 'powrprof.dll',0
1401B95DA                 align 20h
1401B95E0 unk_1401B95E0   db 0FCh                 ; DATA XREF: .data:off_1401C0310↓o
1401B95E1                 db  66h ; f
1401B95E2                 db 0B2h
1401B95E3                 db  70h ; p
1401B95E4                 db 0E8h
1401B95E5                 db  87h
1401B95E6                 db  4Ch ; L
1401B95E7                 db  9Dh
1401B95E8                 db  73h ; s
1401B95E9                 db  4Eh ; N
1401B95EA                 db 0CDh
1401B95EB                 db  5Bh ; [
1401B95EC                 db  76h ; v
1401B95ED                 db  6Ah ; j
1401B95EE                 db 0A5h
1401B95EF                 db  89h
1401B95F0                 db  6Dh ; m
1401B95F1                 db 0DAh
1401B95F2                 db  6Ch ; l
1401B95F3                 db 0C5h
1401B95F4                 db  34h ; 4
1401B95F5                 db  9Dh
1401B95F6                 db  5Fh ; _
1401B95F7                 db  3Bh ; ;
1401B95F8                 db  44h ; D
1401B95F9                 db 0B5h
1401B95FA                 db  4Ah ; J
1401B95FB                 db 0AFh
1401B95FC                 db  5Eh ; ^
1401B95FD                 db 0F9h
1401B95FE                 db 0CEh
1401B95FF                 db  49h ; I
```

```
#sm4

SBOX =
[0xD6,0x90,0xE9,0xFE,0xCC,0xE1,0x3D,0xB7,0x16,0xB6,0x14,0xC2,0x28,0xFB,0x2C,0x05,
0x2B,0x67,0x9A,0x76,0x2A,0xBE,0x04,0xC3,0xAA,0x44,0x13,0x26,0x49,0x86,0x06,0x99,
0x9C,0x42,0x50,0xF4,0x91,0xEF,0x98,0x7A,0x33,0x54,0x0B,0x43,0xED,0xCF,0xAC,0x62,
0xE4,0xB3,0x1C,0xA9,0xC9,0x08,0xE8,0x95,0x80,0xDF,0x94,0xFA,0x75,0x8F,0x3F,0xA6,
0x47,0x07,0xA7,0xFC,0xF3,0x73,0x17,0xBA,0x83,0x59,0x3C,0x19,0xE6,0x85,0x4F,0xA8,
0x68,0x6B,0x81,0xB2,0x71,0x64,0xDA,0x8B,0xF8,0xEB,0x0F,0x4B,0x70,0x56,0x9D,0x35,
0x1E,0x24,0x0E,0x5E,0x63,0x58,0xD1,0xA2,0x25,0x22,0x7C,0x3B,0x01,0x21,0x78,0x87,
0xD4,0x00,0x46,0x57,0x9F,0xD3,0x27,0x52,0x4C,0x36,0x02,0xE7,0xA0,0xC4,0xC8,0x9E,
0xEA,0xBF,0x8A,0xD2,0x40,0xC7,0x38,0xB5,0xA3,0xF7,0xF2,0xCE,0xF9,0x61,0x15,0xA1,
0xE0,0xAE,0x5D,0xA4,0x9B,0x34,0x1A,0x55,0xAD,0x93,0x32,0x30,0xF5,0x8C,0xB1,0xE3,
0x1D,0xF6,0xE2,0x2E,0x82,0x66,0xCA,0x60,0xC0,0x29,0x23,0xAB,0x0D,0x53,0x4E,0x6F,
0xD5,0xDB,0x37,0x45,0xDE,0xFD,0x8E,0x2F,0x03,0xFF,0x6A,0x72,0x6D,0x6C,0x5B,0x51,
0x8D,0x1B,0xAF,0x92,0xBB,0xDD,0xBC,0x7F,0x11,0xD9,0x5C,0x41,0x1F,0x10,0x5A,0xD8,
0x0A,0xC1,0x31,0x88,0xA5,0xCD,0x7B,0xBD,0x2D,0x74,0xD0,0x12,0xB8,0xE5,0xB4,0xB0,
0x89,0x69,0x97,0x4A,0x0C,0x96,0x77,0x7E,0x65,0xB9,0xF1,0x09,0xC5,0x6E,0xC6,0x84,
0x18,0xF0,0x7D,0xEC,0x3A,0xDC,0x4D,0x20,0x79,0xEE,0x5F,0x3E,0xD7,0xCB,0x39,0x48
]
FK = [0xA3B1BAC6,0x56AA3350,0x677D9197,0xB27022DC]
EXTRA = [0xA5A5A5A5,0x3C3C3C3C,0x5A5A5A5A,0xC3C3C3C3]
CK = [
0x00070E15,0x1C232A31,0x383F464D,0x545B6269,0x70777E85,0x8C939AA1,0xA8AFB6BD,0xC4
CBD2D9,
0xE0E7EEF5,0xFC030A11,0x181F262D,0x343B4249,0x50575E65,0x6C737A81,0x888F969D,0xA4
ABB2B9,
```

```
0xC0C7CED5,0xDCE3EAF1,0xF8FF060D,0x141B2229,0x30373E45,0x4C535A61,0x686F767D,0x848B9299,
0xA0A7AEB5,0xBCC3CAD1,0xD8DFE6ED,0xF4FB0209,0x10171E25,0x2C333A41,0x484F565D,0x646B7279
]

def rol(x,n): return ((x<<n)&0xffffffff)|(x>>(32-n))
def tau(a): return (SBOX[(a>>24)&0xff]<<24)|(SBOX[(a>>16)&0xff]<<16)|(SBOX[(a>>8)&0xff]<<8)|SBOX[a&0xff]
def L(b): return b^rol(b,2)^rol(b,10)^rol(b,18)^rol(b,24)
def Lp(b): return b^rol(b,13)^rol(b,23)
def T(x): return L(tau(x))
def Tp(x): return Lp(tau(x))

def key_schedule_mod(mk):
    MK=[int.from_bytes(mk[i:i+4],'big') for i in range(0,16,4)]
    K=[MK[i]^FK[i]^EXTRA[i] for i in range(4)]
    rk=[]
    for i in range(32):
        t=K[i+1]^K[i+2]^K[i+3]^CK[i]
        K.append(K[i]^Tp(t))
        rk.append(K[i+4])
    return rk

def sm4_dec_block(ct, mk):
    rk=key_schedule_mod(mk)[::-1]
    X=[int.from_bytes(ct[i:i+4],'big') for i in range(0,16,4)]
    for i in range(32):
        t=X[i+1]^X[i+2]^X[i+3]^rk[i]
        X.append(X[i]^T(t))
    return b''.join(X[35-j].to_bytes(4,'big') for j in range(4))

# 解密
ct = bytes.fromhex("FC66B270E8874C9D734ECD5B766AA5896DDA6CC5349D5F3B44B54AAF5EF9CE49")
key = bytes.fromhex("26998B3B3292B645F1E3B68D7522A4AD")[::-1]
p = b''.join(sm4_dec_block(ct[i:i+16], key) for i in range(0, len(ct), 16))
print(p.decode())
```